

UNIVERSIDAD EAFIT
INGENIERÍA DE SISTEMAS

DOCUMENTO DE DEFINICIÓN DEL PROYECTO

SPRINT 0 · PROPUESTA DE CAPSTONE PROJECT IN IT

Campo	Información
Asignatura	Capstone Project in IT
Código / grupo	SI4003 - 4875
Profesor	Juan Carlos Montoya Mendoza
Semestre	2026-2
Equipo	Luis Alejandro Castrillón Pulgarín y Sara López Marín
Fecha de entrega	24/08/26

Propósito. Esta propuesta funciona como objeto de aprendizaje, diseño y evaluación. Debe demostrar que existe un problema relevante de IT, que el artefacto propuesto aporta algo más que configuración rutinaria y que su valor podrá comprobarse con evidencia.

VERSIÓN PARA COMPLETAR POR EL EQUIPO

1. Información general del proyecto y del equipo

Campo	Respuesta del equipo
Título provisional	CloudMove: Plataforma de abstracción de arquitectura, migración selectiva de cargas y generación IaC.
Dominio / organización	Caso simulado: Empresas PyME e híbridas con infraestructura on-premise y multicloud, y consultoras IT/MSP que ejecutan migraciones para terceros.
Tipo de proyecto	Plataforma tipo SaaS.
Repositorio	https://github.com/slopma/CapstoneProject_TI
Contacto del stakeholder	Stakeholder simulado: rol de Cloud/DevOps Engineer.

1.1 Equipo y responsabilidades iniciales

Integrante	Correo	Rol inicial	Responsabilidades	Contribución verificable
Sara López Marín	slopezm13@eafit.edu.co	Líder de problema y diseño conceptual	Levantar el contexto, el AS-IS y la evidencia del problema (sección 2); liderar el diseño inicial del modelo CMIR y su representación como grafo (sección 4.3–4.4); coordinar el marco de referencia y la comparación de alternativas (sección 3)	Documento de contexto y AS-IS con fuentes citadas; primer boceto del esquema CMIR (diagrama/documento); tabla de alternativas con criterios definidos
Luis Alejandro Castrillón Pulgarín	lacastrilp@eafit.edu.co	Líder de viabilidad técnica y validación	Definir los requisitos y atributos de calidad medibles (sección 4.5); diseñar el MVP/walking skeleton y el plan de hitos (secciones 5–6); explorar la viabilidad técnica del caso EKS↔AKS (herramientas, APIs, límites de las cuentas sandbox)	Tabla de requisitos con métricas y umbrales; definición del escenario de MVP; nota técnica de viabilidad (qué APIs/herramientas se probaron o revisaron y con qué resultado)

2. Problema, contexto y oportunidad

2.1 Contexto del dominio

Las organizaciones que operan infraestructura híbrida u multicloud enfrentan una fragmentación estructural: cada proveedor (AWS, Azure, GCP) expone sus propios servicios, APIs y modelos de identidad, y la infraestructura on-premise añade una capa adicional de hipervisores, redes y bases de datos autogestionadas. Cuando una empresa decide migrar cargas de trabajo —ya sea por reducción de costos, cumplimiento normativo, resiliencia o para evitar dependencia de un único proveedor (*vendor lock-in*)— debe traducir manualmente arquitecturas completas de un ecosistema a otro.

Las herramientas de migración disponibles hoy como consolas nativas de los proveedores, herramientas de terceros tipo Flexera o Concierto, y motores de generación de IaC como Terraform, resuelven fragmentos del problema pero no lo terminan: los proveedores ofrecen herramientas de migración *nativas* que solo funcionan hacia su propia nube; los motores de FinOps/gestión (Flexera) dan visibilidad de costos pero no generan infraestructura ejecutable; y Terraform o Puppet, usados de forma aislada, requieren que un ingeniero ya conozca y reescriba manualmente el mapeo de un proveedor a otro.

Esto tiene consecuencias medibles: los equipos de DevOps y las consultoras IT/MSP invierten una proporción significativa del tiempo de un proyecto de migración en el descubrimiento manual de la arquitectura existente, en lugar de en la validación funcional de la migración. La ausencia de un modelo intermedio neutral obliga, además, a que cualquier migración parcial, como mover un conjunto de servicios, por ejemplo una API y su caché, dejando el resto en su ubicación original, se resuelva "a mano", identificando dependencias externas de forma manual, lo que introduce alto riesgo de omitir una dependencia crítica de red o de identidad.

Evidencias que dimensionan el problema:

1. La ausencia de un modelo de representación intermedio neutral está documentada como una brecha activa en la arquitectura de referencia de cómputo en la nube publicada por el NIST, que reconoce la portabilidad e interoperabilidad como un desafío no resuelto de forma estándar entre proveedores [1].
2. Las herramientas comerciales de gestión multicloud existentes (p. ej. Flexera, Concierto) se concentran en visibilidad de costos (FinOps) o en orquestación de ejecución, pero ninguna expone o permite editar un modelo semántico intermedio de la arquitectura, ni permite seleccionar y migrar subgrafos parciales de infraestructura (evidencia encontrada por nosotros en la sección 3).

Este contexto define el espacio de oportunidad de CloudMove: no compite por ejecutar mejor una migración monolítica, sino por introducir la capa de abstracción semántica que hoy no existe entre proveedores.

2.2 Situación actual (AS-IS)

Elemento	Situación actual	Evidencia / fuente	Consecuencia
Traducción entre proveedores.	Se realiza manualmente o con scripts ad-hoc de traducción rígida 1-a-1 (ej. EC2 → Compute Engine).	Práctica documentada en herramientas de migración nativa de proveedores.	Cada nueva ruta de migración (AWS→Azure, Azure→GCP, etc.) exige reescritura completa; no hay reutilización del conocimiento de mapeo
Migración de infraestructura.	Se mueven servidores o <i>workloads</i> completos en bloque, no subconjuntos.	Limitación reportada por herramientas tipo <i>lift-and-shift</i> nativas.	Migraciones sobredimensionadas: se mueve infraestructura que no necesitaba migrar, aumentando costo y riesgo
Generación de infraestructura como código.	Los scripts de Terraform se generan de forma estática y aislada, sin gestión de estado continuo.	Documentación oficial de HashiCorp Terraform [2]	El estado de la infraestructura desplegada puede divergir del código sin que exista control automático (<i>configuration drift</i>).

2.3 Resultado deseado (TO-BE)

CloudMove busca que un equipo de DevOps o una consultora IT pueda:

1. Descubrir automáticamente su infraestructura on-premise o cloud existente y representarla en un modelo semántico neutral.
2. Visualizar esa arquitectura como un grafo de dependencias explícitas e implícitas.
3. Seleccionar un subconjunto (subgrafo) de esa arquitectura para migrar, sin tener que mover el resto.
4. Obtener automáticamente el código de infraestructura (Terraform) y las recetas de configuración (Puppet) necesarias para desplegar ese subgrafo en el proveedor destino, manteniendo el estado deseado post-despliegue.
5. Reducción del tiempo de descubrimiento y mapeo manual, y la capacidad de migrar fragmentos de arquitectura con resolución automática de dependencias.

2.4 Stakeholders y usuarios

Actor / usuario	Necesidad o decisión	Interacción con el artefacto	Valor esperado	Criterio de aceptación
Ingeniero DevOps / Cloud Engineer	Migrar una carga de trabajo Kubernetes sin reescribir manifiestos.	Ejecuta Discovery Engine, revisa el grafo, selecciona subgrafo, exporta Terraform/Puppet.	Elimina reescritura manual y reduce riesgo de omitir dependencias.	El código generado despliega el subgrafo seleccionado sin intervención manual adicional.
Consultora IT	Estandarizar y acelerar la fase de descubrimiento y evaluación técnica en proyectos de clientes.	Usa CloudMove como herramienta de diagnóstico y generación de entregables para el cliente.	Reduce tiempo de la fase de discovery en proyectos de migración.	El grafo de arquitectura y el reporte de dependencias se generan sin trabajo manual de inventario.
Empresa PyME sin equipo TI especializado	Migrar a la nube sin contratar especialistas en múltiples proveedores.	Usa la interfaz de visualización y exportación de laC.	Migración asistida y comprensible sin curva de aprendizaje multicloud.	Puede completar una migración guiada por el grafo sin escribir Terraform manualmente.

3. Formulación y sustento del reto de ingeniería

3.1 Enunciado del problema

Los equipos de DevOps y las consultoras IT que operan infraestructura híbrida u multicloud necesitan migrar y evaluar subconjuntos de su arquitectura entre proveedores heterogéneos, porque las herramientas actuales solo traducen de forma rígida y monolítica entre pares fijos de proveedores sin exponer un modelo intermedio neutral, lo cual produce migraciones sobredimensionadas, alto riesgo de omitir dependencias y reescritura manual repetida por cada nueva ruta de migración.

3.2 Pregunta rectora del Capstone

¿Cómo diseñar y validar un modelo de representación intermedio neutral (CMIR) y un mecanismo de selección de subgrafos que permitan migrar fragmentos de arquitectura de infraestructura entre proveedores heterogéneos —incluyendo Kubernetes (EKS↔AKS)— generando IaC ejecutable y manteniendo el estado deseado, bajo las restricciones de un semestre académico y sin reescritura manual de mapeos?

3.3 Marco de referencia y estado de la práctica

Fuente / año	Tipo	Aporte relevante	Limitación / brecha	Implicación para el proyecto
NIST SP 500-292, <i>Cloud Computing Reference Architecture</i> , 2011 [1]	Estándar	Define taxonomía y actores de cómputo en nube, y reconoce la portabilidad/interoperabilidad como reto abierto	No propone un modelo intermedio concreto ni mecanismo de traducción	Justifica la necesidad de un modelo neutral tipo CMIR; sustenta el problema
HashiCorp, <i>Terraform Documentation</i> [2]	Documentación técnica primaria	Define el modelo declarativo de IaC y gestión de estado	No resuelve la traducción semántica entre proveedores; el usuario debe conocer el mapeo	CloudMove usará Terraform como formato de salida, no como motor de traducción
Puppet, <i>Puppet Language and Configuration Guide</i> [3]	Documentación técnica primaria	Modelo de gestión de configuración declarativa y control de estado post-despliegue	No cubre aprovisionamiento o inicial de infraestructura cloud	CloudMove integrará Puppet para mantenimiento continuo del estado, complementando Terraform
CNCF, <i>Cluster API Project Documentation</i> [4]	Documentación técnica primaria / producto	Abstrae el aprovisionamiento de clústeres Kubernetes sobre múltiples proveedores mediante un modelo declarativo común	Limitado al ciclo de vida de clústeres K8s, no cubre cómputo/red/almacenamiento general fuera de K8s	Referente directo para el módulo de conversión EKS↔AKS de CloudMove
OASIS, <i>TOSCA (Topology and Orchestration Specification for Cloud Applications)</i> , estándar [5]	Estándar	Propone un modelo de topología portable para describir aplicaciones cloud de forma independiente del proveedor	Baja adopción práctica en herramientas comerciales; complejidad de implementación	Antecedente conceptual directo del CMIR: valida la idea de un modelo intermedio estándar, pero CloudMove lo simplifica para viabilidad en un semestre

Fuente / año	Tipo	Aporte relevante	Limitación / brecha	Implicación para el proyecto
Flexera, <i>State of the Cloud Report</i> (documentación de producto) [6]	Producto / reporte técnico	Evidencia que la gestión de costos multicloud (FinOps) es la prioridad de mercado actual	No aborda portabilidad estructural ni generación de IaC	Diferenciación: CloudMove no compete en FinOps, compete en portabilidad de arquitectura
G. Pahl et al., "Cloud Container Technologies: A State-of-the-Art Review," <i>IEEE Transactions on Cloud Computing</i> , 2019 [7]	Académica	Revisión del estado del arte en portabilidad de contenedores y Kubernetes entre proveedores	Se enfoca en contenedores, no en infraestructura híbrida completa (VMs, redes, BD)	Sustenta el alcance del caso de uso EKS↔AKS como problema de portabilidad reconocido en literatura
Kubernetes / CNCF, <i>Kubernetes Documentation</i> [8]	Documentación técnica primaria	Define el modelo de objetos y manifiestos Kubernetes usado como formato neutral de salida	Los manifiestos K8s no cubren identidad (IAM) ni red (Ingress) de forma portable entre nubes	Justifica por qué CloudMove debe traducir explícitamente IRSA↔Workload Identity y ALB↔AGIC, no solo los manifiestos

3.4 Alternativas consideradas

Alternativa	Principio de funcionamiento	Ventajas	Limitaciones	Decisión
Mantener el enfoque manual / herramientas nativas del proveedor	Cada migración se planifica y ejecuta a mano usando las consolas y CLIs nativas de origen y destino	Sin curva de aprendizaje de una herramienta nueva; control total	No escala, alto riesgo de omitir dependencias, cada ruta de migración se repite desde cero	Descartar como solución, pero se usa como línea base (AS-IS) de comparación
Herramientas comerciales de gestión multicloud (Flexera, Concierto)	Proveen visibilidad de costos/FinOps y, en algunos casos, orquestación de despliegue sobre múltiples nubes	Maduras, con soporte comercial y ecosistema amplio	No exponen un modelo intermedio editable, no soportan migración selectiva de subgrafos, no generan IaC declarativo versionable	Descartar como sustituto directo; se usan como referencia de mercado en la matriz competitiva

Alternativa	Principio de funcionamiento	Ventajas	Limitaciones	Decisión
Adoptar un estándar existente de topología portable (TOSCA) sin adaptación	Usar la especificación OASIS TOSCA tal como está para modelar la infraestructura	Estándar formal, ya validado por la industria	Complejidad de implementación alta para el alcance de un semestre; baja adopción práctica; no incluye módulo de discovery ni generador Terraform/Puppet integrado	Adoptar el <i>principio</i> (modelo intermedio neutral) pero no la especificación completa; se simplifica en el CMIR propio del equipo

3.5 Brecha y contribución propuesta

Ninguna de las alternativas evaluadas combina las cuatro capacidades que el problema requiere simultáneamente:

1. Un modelo intermedio neutral expuesto y editable.
2. Descubrimiento automatizado de infraestructura híbrida.
3. Selección y migración de subgrafos parciales con resolución automática de dependencias.
4. Generación de IaC declarativo con control de estado continuo.

La contribución de CloudMove es **integrativa y contextual**: no inventa un nuevo estándar de representación, sino que diseña un modelo intermedio (CMIR) deliberadamente acotado al alcance de infraestructura híbrida y Kubernetes, y lo conecta de extremo a extremo con un mecanismo de selección visual de subgrafos y un generador de Terraform + Puppet — algo que hoy exige combinar manualmente varias herramientas que no se hablan entre sí.

4. Objetivos y artefacto de ingeniería

4.1 Objetivo general

Diseñar, construir e integrar CloudMove, una plataforma que descubre infraestructura on-premise y multicloud, la normaliza en un modelo de representación intermedio neutral (CMIR), permite seleccionar subgrafos de arquitectura y genera automáticamente infraestructura como código (Terraform) y configuración de estado (Puppet), y validar que reduce el esfuerzo manual de migración selectiva frente al enfoque de traducción 1-a-1 y migración monolítica actual.

4.2 Objetivos específicos

- **OE1:** Caracterizar el problema de portabilidad de arquitectura entre proveedores heterogéneos y establecer una línea base del esfuerzo manual actual de descubrimiento y mapeo.
- **OE2:** Diseñar el modelo de representación intermedio (CMIR) y el esquema del grafo de dependencias explícitas e implícitas para cómputo, red, almacenamiento y Kubernetes.
- **OE3:** Implementar el Discovery Engine, el motor de selección de subgrafos y el generador de IaC (Terraform + Puppet) para el caso de uso on-premise/AWS/Azure y la conversión EKS↔AKS.
- **OE4:** Diseñar y ejecutar experimentos que comparen el esfuerzo y la exactitud de una migración realizada con CloudMove frente al enfoque manual, usando el caso EKS↔AKS como escenario de validación.
- **OE5:** Documentar la arquitectura, las decisiones de diseño (ADR) y producir una demo reproducible del recorrido de extremo a extremo.

4.3 Definición del artefacto

Dimensión	Definición
Artefacto principal	Plataforma compuesta por: Discovery Engine, motor de normalización semántica (CMIR), grafo interactivo de arquitectura con selector de subgrafos, y generador de IaC (Terraform + Puppet)
Usuarios y operadores	Ingenieros DevOps/Cloud, consultoras TI y equipos técnicos de PyMEs sin especialización multicloud
Entradas y salidas	Entradas: credenciales/acceso de solo lectura a APIs de AWS, Azure y recursos on-premise. Salidas: modelo CMIR (JSON/YAML), grafo visual, módulos Terraform, recetas Puppet
Entorno de ejecución	Laboratorio: clústeres de prueba en cuentas sandbox de AWS y Azure (capa gratuita/créditos académicos) y un entorno on-premise simulado (VMs locales); se justifica por la necesidad de probar migraciones reales sin costo ni riesgo sobre infraestructura productiva
Integraciones	APIs de AWS y Azure, Kubernetes (EKS, AKS), Terraform CLI, Puppet, y agentes <i>agentless</i> de descubrimiento sobre servidores Ubuntu y PostgreSQL
Entregables reproducibles	Código fuente del Discovery Engine y parsers, esquema CMIR documentado, módulos Terraform y recetas Puppet generados de ejemplo, ADRs, suite de pruebas, manual de despliegue del caso EKS↔AKS

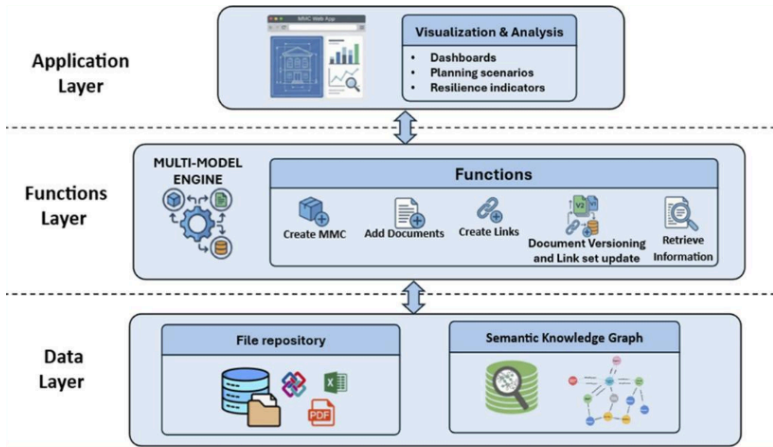
4.4 Arquitectura conceptual inicial

La arquitectura conceptual de CloudMove se organiza en tres capas. La **capa de datos** contiene el repositorio de archivos (credenciales, exportaciones, manifiestos) y el grafo de conocimiento semántico, que es la representación persistente del CMIR. La **capa de funciones** aloja el motor multi-modelo, responsable de las operaciones centrales: creación del modelo CMIR a partir de lo descubierto, adición de documentos/recursos, creación de enlaces (dependencias) entre nodos, versionado del modelo y recuperación de información para consultas del usuario. La **capa de aplicación** expone la interfaz de visualización (grafo interactivo, dashboards, indicadores de resiliencia) con la que interactúan los usuarios finales.

El flujo de extremo a extremo atraviesa estas capas en cuatro etapas:

1. El **Discovery Engine**, en el límite del sistema hacia el entorno externo, recolecta infraestructura vía APIs de proveedores y agentes *agentless* sobre recursos on-premise.
2. La **Normalization Layer** traduce esos recursos heterogéneos a nodos semánticos neutrales del CMIR.
3. El **Architecture Graph** calcula dependencias implícitas y explícitas entre nodos y permite al usuario seleccionar un subgrafo.

4. El **Target Parser & IaC Generator** traduce el sub



Terraform y recetas Puppet para el proveedor destino.

grafo seleccionado a módulos

Espacio para el diagrama.

4.5 Requisitos y atributos de calidad

ID	Requisito / escenario	Atributo	Métrica	Umbral objetivo	Método de verificación
RQ-01	Ante la selección de un subgrafo de arquitectura por el usuario, el sistema deberá calcular automáticamente las dependencias externas requeridas antes de generar IaC	Correctitud funcional	% de dependencias externas correctamente identificadas	≥ 90% sobre un set de prueba con dependencias conocidas	Experimento controlado con escenarios de arquitectura etiquetados manualmente
RQ-02	Ante una arquitectura Kubernetes EKS descubierta, el sistema deberá generar manifiestos y Terraform desplegados en AKS sin intervención manual	Portabilidad	% de recursos traducidos automáticamente (EBS CSI, IRSA, ALB Ingress) sin edición manual	≥ 80% de los recursos del caso de prueba	Despliegue real en clúster AKS sandbox y verificación de estado
RQ-03	Ante un cambio en el estado desplegado (deriva de configuración), Puppet deberá detectarlo y reportarlo	Mantenibilidad / control de estado	Tiempo de detección de deriva	≤ 15 minutos desde la introducción del cambio	Prueba controlada: modificar un recurso desplegado y medir tiempo de detección en el reporte de Puppet

ID	Requisito / escenario	Atributo	Métrica	Umbral objetivo	Método de verificación
RQ-04	Ante el escaneo de un entorno on-premise/AWS de tamaño de referencia, el Discovery Engine deberá completar el modelo CMIR en un tiempo acotado	Rendimiento	Tiempo total de descubrimiento y normalización	≤ 10 minutos para un entorno de referencia de ~30 recursos	Medición directa en entorno de laboratorio, 5 repeticiones

5. Alcance, validación y evidencia

5.1 Alcance

Incluye	No incluye	Supuestos	Restricciones
Discovery Engine (AWS, Azure, On-Premise); motor CMIR (cómputo, red, storage, K8s); grafo con selección de subgrafos; generador Terraform + Puppet; caso de uso EKS↔AKS	Replicación continua zero-downtime; refactorización de código fuente de aplicaciones; orquestadores serverless propietarios (AWS Step Functions); hipervisores secundarios; soporte GCP (queda en visión extendida, fuera del MVP del semestre)	Se dispone de cuentas sandbox/académicas en AWS y Azure; el entorno on-premise se simula con VMs locales; el equipo tiene acceso a créditos cloud suficientes para las pruebas	Duración de un semestre (14 semanas); dos integrantes; sin presupuesto para licencias comerciales; alcance de discovery limitado a servidores Ubuntu/RHEL y PostgreSQL

5.2 Hipótesis y criterios de éxito

Hipótesis	Línea base / comparación	Métrica	Diseño de prueba	Criterio de éxito	Evidencia
Si se introduce un modelo intermedio neutral (CMIR) con selección de subgrafos, entonces el esfuerzo manual de mapeo de dependencias se reduce frente al enfoque de traducción 1-a-1 manual	Mapeo manual de dependencias por un ingeniero sobre el mismo escenario	Tiempo y número de dependencias omitidas	Mismo escenario de arquitectura resuelto manualmente vs. con CloudMove, 3 repeticiones	Reducción de tiempo ≥ 50% y cero dependencias críticas omitidas con CloudMove	Registro de tiempos y logs de ejecución del subgraph picker

Hipótesis	Línea base / comparación	Métrica	Diseño de prueba	Criterio de éxito	Evidencia
Si CloudMove traduce automáticamente IRSA→Workload Identity, EBS CSI→Azure Disk y ALB→AGIC, entonces un workload EKS puede desplegarse en AKS sin reescritura manual de manifiestos	Traducción manual de estos tres componentes por un ingeniero	% de manifiestos que requieren edición manual posterior	Despliegue del mismo workload en clúster AKS sandbox, con y sin CloudMove	≤ 20% de manifiestos requieren edición manual con CloudMove	Manifiestos generados, logs de despliegue, estado del clúster
Si Puppet gestiona el estado post-despliegue, entonces la deriva de configuración se detecta en un tiempo acotado sin monitoreo manual	Sin herramienta de control de estado, la deriva se detecta solo en incidentes	Tiempo de detección de deriva	Introducir cambio controlado y medir tiempo hasta el reporte de Puppet	≤ 15 minutos	Reporte Puppet, logs de tiempo

5.3 Producto mínimo demostrable (MVP / walking skeleton)

- **Escenario de demostración:** un clúster EKS de referencia (con un servicio backend + una caché Redis + su almacenamiento persistente) es descubierto por CloudMove, normalizado a CMIR, visualizado como grafo, y el usuario selecciona el subgrafo backend+caché. CloudMove resuelve las dependencias externas necesarias (red, identidad) y genera el Terraform + manifiestos K8s + receta Puppet para desplegar ese subgrafo en un clúster AKS sandbox.
- **Componentes indispensables:** Discovery Engine mínimo para EKS, esquema CMIR básico (cómputo + red + K8s), algoritmo de resolución de dependencias sobre el subgrafo, parser EKS→AKS para los tres mapeos críticos (EBS CSI, IRSA, ALB Ingress), generador de Terraform.
- **Datos/entradas:** credenciales de solo lectura al clúster EKS sandbox del equipo.
- **Resultado observable:** el subgrafo seleccionado queda desplegado y en estado Ready en el clúster AKS destino, con Puppet reportando el estado inicial correcto.
- **Evidencia que se capturará:** logs de descubrimiento, captura del grafo con el subgrafo resaltado, código Terraform generado, salida de `kubectl get pods` en el clúster destino, primer reporte de estado de Puppet.

6. Método de trabajo y plan incremental

El equipo combinará diseño de ingeniería, construcción incremental y evaluación empírica. Scrum puede aportar cadencia, pero no sustituye el método técnico. Las decisiones relevantes se registrarán mediante Architecture Decision Records (ADR); el repositorio, la automatización y la evidencia experimental formarán parte del producto.

6.1 Fases de trabajo

Fase	Pregunta dominante	Actividades	Evidencia / salida
1. Descubrimiento	¿Existe y cómo se manifiesta el problema?	Contexto, stakeholders, AS-IS, línea base, restricciones.	Problema sustentado y datos de línea base.
2. Referentes y alternativas	¿Qué se sabe y qué opciones existen?	Literatura, estándares, arquitecturas, productos y trade-offs.	Síntesis crítica y decisión argumentada.

Fase	Pregunta dominante	Actividades	Evidencia / salida
3. Diseño	¿Qué artefacto puede cerrar la brecha?	Requisitos, arquitectura, ADR, modelo de amenazas y plan de pruebas.	Diseño trazable y backlog priorizado.
4. Construcción incremental	¿Puede operar de extremo a extremo?	Walking skeleton, automatización, integración y observabilidad.	Incrementos reproducibles y demostrables.
5. Evaluación	¿Cumple las hipótesis y bajo qué límites?	Experimentos, comparación, análisis de resultados y amenazas a la validez.	Evidencia, resultados y discusión.
6. Transferencia	¿Puede otro equipo comprenderlo y reproducirlo?	Documentación, empaquetado, demo y recomendaciones.	Producto final, manuales y sustentación.

6.2 Plan de hitos

Hito	Semanas sugeridas	Resultado demostrable	Decisión de avance
Sprint 0 · Propuesta	4-7	Problema, referentes iniciales, alternativas, artefacto, arquitectura conceptual y validación.	¿Es relevante, viable y evaluable?
Sprint 1 · Walking skeleton	7-10	Recorrido mínimo de extremo a extremo y prueba del mayor riesgo técnico.	¿La arquitectura básica funciona?
Sprint 2 · Producto beta	11-15	Capacidades centrales, automatización, pruebas y primer conjunto de mediciones.	¿Existe evidencia de valor y calidad?
Sprint final · Evaluación	16-17	Artefacto completo, experimentos, resultados, límites, documentación y demo.	¿La contribución está sustentada?

6.3 Backlog inicial

ID	Resultado / historia	Criterio de aceptación	Dependencias	Hito	Responsable
PB-01	Como ingeniero DevOps, quiero que el Discovery Engine descubra un clúster EKS de referencia	El CMIR resultante contiene todos los recursos del clúster de prueba con sus relaciones	Acceso sandbox a AWS	Sprint 1	Luis
PB-02	Como ingeniero DevOps, quiero visualizar la arquitectura descubierta como un grafo interactivo	El grafo renderiza nodos y dependencias correctamente para el escenario de prueba	PB-01	Sprint 1	Sara
PB-03	Como usuario, quiero seleccionar un subgrafo y que se resuelvan sus dependencias externas automáticamente	El sistema lista correctamente las dependencias externas del subgrafo seleccionado	PB-02	Sprint 2	Sara

ID	Resultado / historia	Criterio de aceptación	Dependencias	Hito	Responsable
PB-04	Como ingeniero DevOps, quiero exportar el subgrafo seleccionado como Terraform + manifiestos K8s desplegables en AKS	El código generado despliega sin errores en el clúster AKS sandbox	PB-03	Sprint 2	Luis
PB-05	Como operador, quiero que Puppet reporte deriva de configuración sobre lo desplegado	Un cambio introducido manualmente se refleja en el reporte de Puppet en ≤15 min	PB-04	Sprint final	Luis y Sara

7. Riesgos, ética, seguridad y sostenibilidad

Riesgo	Causa	Prob. / impacto	Indicador temprano	Mitigación	Contingencia	Responsable
Créditos/cuota de las cuentas sandbox de AWS/Azure se agotan antes de terminar las pruebas	Múltiples ciclos de despliegue/destrucción de clústeres EKS/AKS durante desarrollo	Media/Alta	Alertas de consumo de créditos	Automatizar destrucción de recursos tras cada prueba; usar clústeres pequeños	Migrar pruebas a entornos locales simulados (kind/minikube)	Luis
El mapeo semántico CMIR resulta insuficiente para capturar todas las dependencias reales de un caso complejo	Complejidad subestimada del modelo de dependencias implícitas	Media/Media	Casos de prueba con dependencias no detectadas en RQ-01	Iterar el esquema CMIR con casos de prueba progresivamente más complejos	Acotar el alcance del MVP a las dependencias explícitas primero	Sara
Exposición de credenciales cloud durante el desarrollo del Discovery Engine	Manejo de credenciales de solo lectura en un repositorio de código compartido	Baja/Alta	Escaneo de secretos en commits	Uso de variables de entorno y <code>.gitignore</code> ; nunca credenciales en el repo	Rotación inmediata de credenciales si se detecta exposición	Ambos

Riesgo	Causa	Prob. / impacto	Indicador temprano	Mitigación	Contingencia	Responsable
El semestre no alcanza para cubrir el conversor completo EKS↔AKS con los tres mapeos críticos	Subestimación del esfuerzo de integración con IAM/identidad entre proveedores	Media/ Alta	Retraso en PB-04 al cierre del Sprint 2	Priorizar los tres mapeos críticos (EBS, IRSA, ALB) sobre cobertura amplia	Reducir el caso de demo a solo cómputo + red, dejando identidad como trabajo futuro	Luis

8. Referencias

- [1] F. Liu, J. Tong, J. Mao, R. Bohn, J. Messina, M. Badger, and D. Leaf, *NIST Cloud Computing Reference Architecture*, NIST Special Publication 500-292, National Institute of Standards and Technology, Gaithersburg, MD, 2011. [Online]. Available: <https://doi.org/10.6028/NIST.SP.500-292>
- [2] HashiCorp, "Terraform Documentation." [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
- [3] Puppet, "Puppet Language and Configuration Guide." [Online]. Available: <https://www.puppet.com/docs>
- [4] Cloud Native Computing Foundation, "Cluster API Project Documentation." [Online]. Available: <https://cluster-api.sigs.k8s.io/>
- [5] OASIS, *Topology and Orchestration Specification for Cloud Applications (TOSCA) Version 1.0*, OASIS Standard, 2013.
- [6] Flexera, "State of the Cloud Report." [Online]. Available: <https://www.flexera.com/>
- [7] G. Pahl, A. Brogi, J. Soldani, and P. Jamshidi, "Cloud Container Technologies: A State-of-the-Art Review," *IEEE Transactions on Cloud Computing*, vol. 7, no. 3, 2019.
- [8] Cloud Native Computing Foundation, "Kubernetes Documentation." [Online]. Available: <https://kubernetes.io/docs/>

9. Lista de verificación antes de entregar

- El problema está sustentado con evidencia y no está redactado como una solución.
- La contribución va más allá de configurar o desplegar herramientas existentes.
- Se compararon alternativas y se justificó la decisión inicial.
- El artefacto, sus usuarios, límites e integraciones están claramente definidos.
- Los atributos de calidad tienen métricas, umbrales y métodos de verificación.
- El MVP reduce el mayor riesgo técnico y funciona de extremo a extremo.
- El plan cabe en el semestre y explicita qué no se construirá.
- Las referencias están citadas y el repositorio contiene la evidencia requerida.

10. Rúbrica de Sprint 0 · Propuesta

La propuesta se evalúa por la calidad del razonamiento y la trazabilidad entre problema, referentes, artefacto y validación. Una presentación tecnológica atractiva no compensa un problema débil o no evaluable.

Criterio	Peso	Sobresaliente	Aceptable	Insuficiente
Problema, contexto y evidencia	20%	Problema específico, relevante y cuantificado; línea base y stakeholders coherentes.	Problema comprensible, pero con evidencia parcial o línea base débil.	Problema genérico, tecnológico o sin evidencia.
Referentes, alternativas y brecha	20%	Síntesis crítica de fuentes sólidas; alternativas comparadas y brecha defendible.	Fuentes y alternativas presentes, con análisis limitado.	Lista de tecnologías sin análisis ni brecha.
Contribución y artefacto	20%	Artefacto claro, no trivial, con límites, usuarios, integraciones y entregables reproducibles.	Artefacto viable, pero ambiguo en alcance o contribución.	Aplicación/despliegue rutinario o artefacto indefinido.
Arquitectura y requisitos	15%	Arquitectura conceptual propia; requisitos trazables y atributos de calidad medibles.	Arquitectura y requisitos parciales o poco conectados.	Diagrama decorativo y requisitos no verificables.
Validación y factibilidad	15%	Hipótesis refutables, comparación, métricas, MVP y plan viables.	Plan plausible, pero métricas o línea base incompletas.	No hay forma clara de demostrar resultados.
Comunicación y rigor	10%	Documento preciso, coherente, bien citado y profesional.	Comprensible, con inconsistencias menores.	Incompleto, desarticulado o sin referencias adecuadas.